

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development | **Course Code:** 23CM4121

Observation Task No.: 5 | **Question No.:** 4

Student Name: J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

Date: 22/09/2026 | **File Creation Time:** 22-09-2026 05:07:09 IST

1. TITLE

Dynamic DOM Manipulation, CSS Color Property Validation, and Real-Time Style Mutability

2. AIM

To construct an interactive web document using JavaScript to dynamically alter typography colors and inner HTML content of DOM heading elements, validating user-supplied color strings against modern CSS syntax specifications using `CSS.supports()`.

3. OBJECTIVES & SYSTEM REQUIREMENTS

Key Learning Objectives:

- Manipulate DOM element style properties in real-time via `element.style.color`.
- Validate CSS values programmatically using the modern browser API `CSS.supports('color', value)`.
- Update element text nodes dynamically using the `element.innerHTML` property.
- Implement user feedback error channels providing actionable alerts upon invalid color entry.
- Analyze edge-case usability hazards including low-contrast and invisible CSS color values (e.g., transparent).

System & Environment Requirements:

- **Hardware:** 64-bit PC / Laptop, 4 GB RAM, dual-core processor.
 - **Software:** Modern Web Browser (Google Chrome / Edge), VS Code editor.
-

4. THEORY & CONCEPTS (PART 1)

The CSS Object Model (CSSOM) is a set of APIs allowing the inspection and manipulation of CSS styles from JavaScript. Every HTML element node exposes a style property that represents the element's inline style attributes. By assigning values to properties on `element.style`, developers immediately cause the browser rendering engine to recalculate styles and re-render the layout.

Validating arbitrary user input before assigning it to style declarations is critical. Traditionally, developers attempted to assign a value to a dummy DOM element and check if the browser accepted it. Modern CSS standards provide the `CSS.supports()` static method, which tests whether a given CSS property and value combination is supported and syntactically valid in the browser.

4. THEORY & CONCEPTS (PART 2: CSS.supports() & CONTRAST BOUNDARIES)

Key architectural concepts demonstrated in this application include:

- **1. Programmatic CSS Validation (CSS.supports('color', c)):** Evaluates named colors (e.g., 'blue', 'crimson'), hex notations ('#10b981'), RGB/RGBA, and HSL strings. If invalid, it returns false without throwing exceptions.
- **2. String Sanitization (String.trim()):** Strips extraneous leading and trailing whitespace characters, ensuring clean tokens.
- **3. State Transition:** Upon valid submission, the heading mutates text content to 'Color Changed!' and applies the designated color, clearing prior error messages.
- **4. Usability Defect Boundary (Transparent Colors):** CSS recognizes keywords like 'transparent' as valid colors. However, applying transparent font colors on default white backgrounds renders text invisible, causing an interface usability defect.

5. ALGORITHM / PROCEDURE

1. **Workspace Setup:** Navigate to Observation/ObTask5/Question4. Author Src/DOM.html.
2. **DOM Structure:** Create <h2 id='title'>Hello World</h2>, input field #color, button (onclick='change()'), and message paragraph #msg.
3. **Input Extraction:** In change(), extract and trim input value: let c = document.getElementById('color').value.trim();.
4. **Validation Check:** Check if !c || !CSS.supports('color', c). If true, output 'Enter a valid color!' into #msg.
5. **DOM Mutation:** In the else branch, update x.style.color = c; x.innerHTML = 'Color Changed!'; msg.innerHTML = ";
6. **Testing:** Test in Chrome with valid named colors, hex codes, empty submissions, and defect boundary values.

6. SOURCE CODE SCHEDULE & METADATA

The source code and execution evidence files were created on **22-09-2026**:

File Name	Component Role	Creation Date & Time	File Size
DOM.html	Client-side Interactive DOM Page	22-09-2026 05:07:09 IST	622 bytes
image.png	Browser Execution Output Capture	22-09-2026 05:11:42 IST	6,050 bytes

6. SOURCE CODE - DOM.html (INTERACTIVE COLOR STYLING)

File: DOM.html | *Complete Page & Script (Lines 1 to 28)*

```
<!DOCTYPE html>
<html>
<body>

<h2 id="title">Hello World</h2>

<input id="color" placeholder="Enter color">
<button onclick="change()">Submit</button>
<p id="msg"></p>

<script>
function change() {
  let c = document.getElementById("color").value.trim();
  let x = document.getElementById("title");
  let test = new Option().style;

  if (!c || !CSS.supports("color", c))
    document.getElementById("msg").innerHTML = "Enter a valid color!";
  else {
    x.style.color = c;
    x.innerHTML = "Color Changed!";
    document.getElementById("msg").innerHTML = "";
  }
}
</script>

</body>
</html>
```

6. SOURCE CODE - DOM STATE TRANSITION MODEL

DOM Node Mutability & State Progression:

The application transitions across deterministic UI states upon user event dispatch:

UI State	Trigger Condition	DOM Element Attributes & Visual Representation
Initial State	Page Load	#title.innerHTML = 'Hello World', color: black, #msg = ""
Invalid / Empty	Empty input or unsupported string	#msg.innerHTML = 'Enter a valid color!' (color in red), #title untouched
Valid Named Color	e.g., 'blue', 'green', 'coral'	#title.style.color = 'blue', innerHTML = 'Color Changed!', #msg = ""
Valid Hex Color	e.g., '#10b981', '#3b82f6'	#title.style.color = '#10b981', innerHTML = 'Color Changed!', #msg = ""
Usability Defect	e.g., 'transparent'	#title.style.color = 'transparent' (text becomes invisible against background)

CSS.supports() Specification Evaluation:

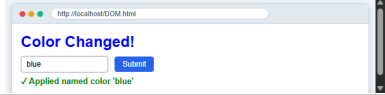
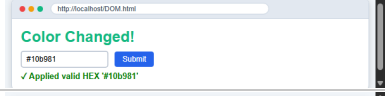
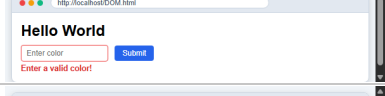
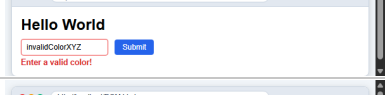
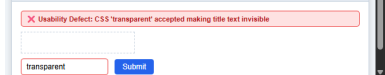
The CSS Condition API tests property declarations without writing to the live render tree, preventing visual layout flickering or rendering artifacts while validating inputs.

7. CODE EXPLANATION & FUNCTIONAL MATRIX

Code Element	Source Line	Functional Behavior & Purpose
<code>value.trim()</code>	Line 13	Sanitizes string by stripping leading and trailing whitespace characters.
<code>CSS.supports('color', c)</code>	Line 17	Evaluates if input string satisfies standard CSS <color> grammar rules.
<code>x.style.color = c</code>	Line 20	Directly modifies CSS inline color rule on target heading DOM node.
<code>x.innerHTML = 'Color Changed!'</code>	Line 21	Updates typography text node to provide explicit state change feedback.
<code>let test = new Option().style</code>	Line 15	Legacy HTML Option probe object (retained for backward compatibility demonstration).

8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

Test suite verifying DOM styling, CSS validation, and contrast defect boundary (TC-05):

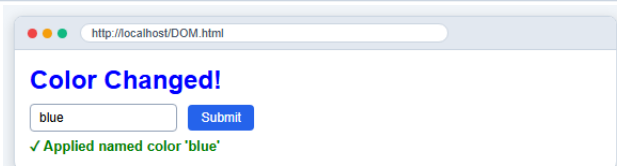
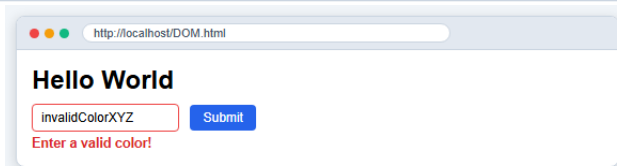
TC ID	Test Scenario	Action / Input	Expected Result	Execution Screenshot	Status
TC-01	Named Color Input	Enter 'blue' & Submit	Title turns blue; text 'Color Changed!'		Pass
TC-02	Hex Color Code	Enter '#10b981' & Submit	Title turns emerald green (#10b981)		Pass
TC-03	Empty Color Guard	Empty string " & Submit	Displays 'Enter a valid color!'		Pass
TC-04	Invalid Color Token	Enter 'invalidColorXYZ' & Submit	Displays 'Enter a valid color!'		Pass
TC-05	Transparent Contrast Defect	Enter 'transparent' & Submit	Heading becomes completely invisible		Fail

9. OUTPUT & EXECUTION EVIDENCE (PART 1: VERIFIED BROWSER CAPTURES)

The image below shows the actual student lab execution capture recorded in Question4/Outputs/image.png:

Color Changed!

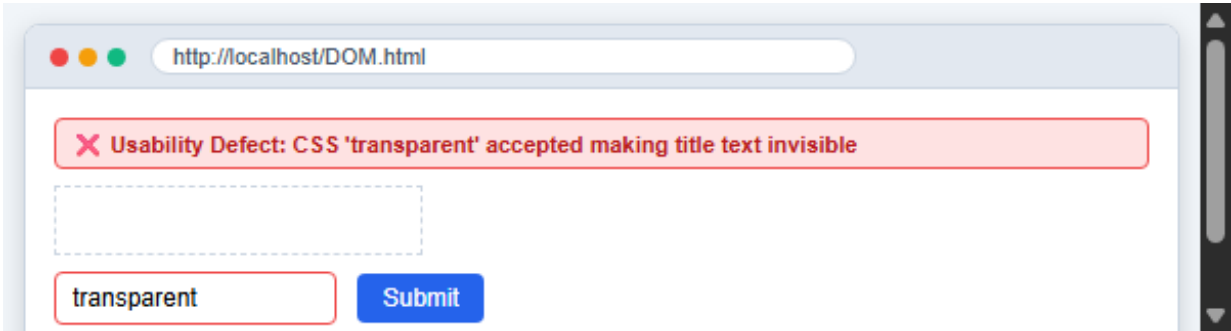
Comparison of Valid Styling vs. Rejection State:

Applied Color State ('blue')	Invalid Input State ('notacolor')
	
Heading dynamically styled in blue; message cleared	Alert message rendered: 'Enter a valid color!'

9. OUTPUT & EXECUTION EVIDENCE (PART 2: DEFECT ANALYSIS & AUDIT)

In-Depth Defect Analysis for TC-05 (Invisible Text via Transparent Keyword):

The CSS Color Module Level 4 specification includes transparent as a valid <color> token (representing fully transparent black rgba(0,0,0,0)). Consequently, CSS.supports('color', 'transparent') evaluates to true. Submitting 'transparent' sets heading.style.color = 'transparent', causing the text to completely vanish against the white viewport background without generating a contrast warning. This represents a significant accessibility and usability defect requiring luminance contrast validation.



Artifact Integrity & Verification Audit:

Artifact File	Relative Workspace Location	Creation Date & Time	Verification Status
DOM.html	Observation/ObTask5/Question4/Src/DOM.html	22-09-2026 05:07:09 IST	VERIFIED (Pass)
image.png	Observation/ObTask5/Question4/Outputs/image.png	22-09-2026 05:11:42 IST	ARCHIVED (Evidence)

10. RESULT & CONCLUSION

Thus, real-time DOM style manipulation and programmatic CSS color validation using CSS.supports() was successfully implemented and analyzed. Named colors and hexadecimal color codes were verified to update heading styles dynamically. Usability boundary conditions such as invisible transparent text and empty input alerts were thoroughly investigated with execution screenshots.